Beginner's Guide to Docker

From Zero to Containerized — Step by Step

Created by Yogesh Chavan

yogeshchavan.dev

https://www.linkedin.com/in/yogesh-chavan97/

Build · Ship · Run — Anywhere

Table of Contents

Section 1: Introduction to Docker 5

- What is Docker? 5
- Why Use Docker? 5
- Containers vs Virtual Machines 5
- Core Docker Concepts 6
- How Docker Works (Architecture) 6
- Installing Docker 6
  - On macOS and Windows 6
  - On Linux (Ubuntu/Debian) 6
- Verifying the Installation 7
- Running Your First Container 7

Section 2: Docker Images 9

- What Is an Image? 9
- Pulling Images from Docker Hub 9
- Listing and Removing Images 9
  - Fixing the "Image Is Being Used" Error 10
  - Option 1: Remove the container, then the image (recommended) 10
  - Option 2: Force-remove the image 11
- Image Tags and Versions 11
- Searching the Registry 11
- Image Layers Explained 12

Section 3: Working with Containers 13

- Starting Containers (docker run) 13
- Detached Mode and Naming 13
- Listing Containers (ps) 14
- Stopping, Starting, Restarting 14
- Removing Containers 15
- Executing Commands Inside a Container 15
- Viewing Logs 15
- Inspecting Containers 16

Section 4: Writing Dockerfiles 17

- What Is a Dockerfile? 17
- Anatomy of a Dockerfile 17
- Common Instructions Explained 17
- Your First Dockerfile (Node.js App) 18

Step 1: Create the project folder 18
Step 2: Create the package.json file 18
Step 3: Create the server.js file 19
Step 4: Create the Dockerfile 19
Step 5: Create the .dockerignore file 20
Building the Image 20
Updating Your Code: Do You Need to Rebuild? 21
The standard workflow: rebuild and re-run 21
The development shortcut: bind mount the source code 22
The Build Cache and Layer Ordering 22
Bad: invalidates cache on every code change 22
Good: dependencies cached separately from source 23
.dockerignore 23
Why Is .env Excluded? Doesn't the App Need It? 23
Multi-Stage Builds 24

Section 5: Data Persistence with Volumes 25

Why Containers Are Ephemeral 25
Bind Mounts vs Named Volumes 25
Creating and Using Volumes 25
Bind Mounts (Development) 25
Named Volumes (Production) 26
Inspecting and Removing Volumes 26
Real-World Example: Postgres Volume 26
Step 1: Create a named volume 26
Step 2: Start the Postgres container 26
Step 3: Connect and create some data 27
Viewing the Data via a Shell Inside the Container 27
Step 4: Destroy the container 28
Step 5: Start fresh and confirm the data survived 28
Cleaning up 29

Section 6: Docker Networking 30

Default Networks 30
Creating a User-Defined Bridge 30
Connecting Containers by Name 30
Publishing Ports 31
Inspecting Networks 31

Section 7: Docker Compose 33

Why Compose? 33
Installing Docker Compose 33
Your First docker-compose.yml 33
Starting and Accessing the Application 34

Common Compose Commands 35
Real Example: Node.js + Postgres + Redis 35
Environment Variables and .env Files 36

Section 8: Best Practices & Next Steps 38

Image Size Optimization 38
Security Tips 38
Tagging and Versioning Strategy 39
Where to Go From Here 39

1. Introduction to Docker

What is Docker?

Docker is an open-source platform that allows developers to build, ship, and run applications inside lightweight, isolated environments called **containers**. A container packages everything an application needs to run — code, runtime, system libraries, and configuration — into a single portable unit. This means an application that runs on your laptop will run identically on a colleague's machine, a staging server, or a production cloud instance, eliminating the classic `it works on my machine` problem.

Originally released in 2013 by Solomon Hykes, Docker has become the de facto standard for containerization. It builds on long-standing Linux features (namespaces and cgroups) but wraps them in a developer-friendly toolchain with a simple command-line interface, a public image registry called Docker Hub, and a declarative file format for describing how images should be built.

Why Use Docker?

Before Docker, deploying applications meant carefully matching server environments, installing the right versions of language runtimes, system libraries, and OS patches. A single mismatch could break production. Docker solves this and brings several other benefits:

- **Consistency across environments** — the same image runs the same on every machine.
- **Lightweight and fast** — containers start in seconds and use a fraction of the resources of virtual machines.
- **Isolation** — each container has its own filesystem, processes, and network stack.
- **Reproducibility** — a Dockerfile is a recipe; anyone can rebuild the exact same image.
- **Microservices-friendly** — small, single-purpose containers compose into larger systems easily.
- **Massive ecosystem** — Docker Hub hosts hundreds of thousands of ready-to-use images.

Containers vs Virtual Machines

Both containers and virtual machines provide isolation, but they work very differently. A virtual machine includes a full guest operating system on top of a hypervisor, while a container shares the host operating system's kernel and isolates only the application and its dependencies.

Feature	Virtual Machine	Docker Container
Boot time	Minutes	Seconds
Size on disk	Gigabytes	Megabytes
OS overhead	Full guest OS	Shares host kernel
Isolation level	Hardware-level	Process-level
Performance	Lower (virtualized)	Near-native
Portability	Limited	Excellent

> **Note:** Containers do not replace virtual machines in every situation. VMs are still preferable when you need to run a completely different operating system or require strict hardware-level isolation.

Core Docker Concepts

Before you start using Docker, it helps to understand the vocabulary. These five concepts come up constantly:

- **Image** — a read-only template containing your application and its dependencies. Think of it like a class in OOP — a blueprint.
- **Container** — a running instance of an image. You can have many containers from one image, like many objects from one class.
- **Dockerfile** — a text file with step-by-step instructions for building an image.
- **Registry** — a server that stores and distributes images. Docker Hub is the default public registry.
- **Volume** — a mechanism for persisting data outside a container's lifecycle.

How Docker Works (Architecture)

Docker uses a client-server architecture. When you type a command like `docker run nginx`, the following happens behind the scenes:

- The **Docker CLI** (the client) sends your command to the **Docker Daemon** (`dockerd`) over a REST API.
- The daemon checks whether the requested image exists locally; if not, it pulls it from a registry such as Docker Hub.
- The daemon creates a container from the image, allocates a writable filesystem layer, attaches networking, and starts the process.
- Output streams back to your terminal through the daemon.

> **Note:** On Linux, the daemon talks directly to the kernel. On macOS and Windows, Docker Desktop runs a tiny Linux VM under the hood — but you barely notice.

Installing Docker

On macOS and Windows

The easiest path is to install **Docker Desktop**, an all-in-one bundle that includes the daemon, CLI, Docker Compose, and a graphical interface for managing images and containers.

- Visit https://www.docker.com/products/docker-desktop
- Download the installer for your operating system
- Run the installer and accept the default options
- Launch Docker Desktop — you should see a whale icon in your menu bar / system tray

On Linux (Ubuntu/Debian)

On Linux you install the Docker Engine directly using your package manager:

```
# Update package index
sudo apt-get update

# Install prerequisites
sudo apt-get install ca-certificates curl gnupg

# Add Docker's official GPG key
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
    | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

# Add the Docker repository
echo "deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.gpg] \
    https://download.docker.com/linux/ubuntu jammy stable" \
    | sudo tee /etc/apt/sources.list.d/docker.list

# Install Docker Engine
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io
```

> **Tip:** After installation on Linux, add your user to the `docker` group so you don't need `sudo` for every command: `sudo usermod -aG docker $USER`, then log out and back in.

Verifying the Installation

Once installed, verify everything is working by checking the version:

```
# Check Docker version
docker --version
# Output: Docker version 25.0.0, build ...

# Check the daemon and CLI status together
docker info

# Run the official "hello-world" image
docker run hello-world
```

The `hello-world` image is tiny (a few kilobytes) and prints a friendly message confirming that the daemon, CLI, image pulling, and container execution all work.

Running Your First Container

Let's run something more interesting — an Nginx web server — in a single command:

```
# Run nginx in the background and map port 8080 -> 80
docker run -d -p 8080:80 --name my-nginx nginx

# Open http://localhost:8080 in your browser
# You should see the Nginx welcome page

# When you are done, stop and remove the container
docker stop my-nginx
docker rm my-nginx
```

Let's break that command down piece by piece:

- `docker run` — create and start a new container from an image.
- `-d` — **detached** mode; run in the background.
- `-p 8080:80` — publish container port 80 to host port 8080.
- `--name my-nginx` — give the container a friendly name.
- `nginx` — the image to run (pulled from Docker Hub if not present).

> **Best Practice:** Always give your containers explicit names with `--name`. Without it, Docker assigns a random name like `elated_einstein`, which makes them harder to manage in scripts.

2. Docker Images

What Is an Image?

A Docker image is a read-only, layered filesystem snapshot that contains everything needed to run a piece of software: the operating system base, language runtime, application code, dependencies, and metadata such as the default command and exposed ports. Images are immutable — once built, they don't change. To update one, you build a new version and tag it.

Images are identified by a **name** and a **tag**, written together as `name:tag`. For example, `node:20-alpine` refers to the Node.js image at version 20, built on the small Alpine Linux base. If you omit the tag, Docker assumes `:latest`.

Pulling Images from Docker Hub

Docker Hub is the world's largest public registry of container images. You can pull any public image with a single command:

```
# Pull the latest version of an image
docker pull nginx

# Pull a specific version (tag)
docker pull nginx:1.25-alpine

# Pull a particular Node.js LTS image
docker pull node:20-alpine

# Pull the official Postgres image
docker pull postgres:16
```

> **Note:** Tags like `alpine` usually mean the image is built on Alpine Linux, which is much smaller (often under 10 MB) than a Debian or Ubuntu base. Prefer Alpine variants when image size matters.

Listing and Removing Images

You can see all images stored locally with the following commands:

```
# List all images
docker images

# Same thing, newer syntax
docker image ls

# Example output:
# REPOSITORY    TAG          IMAGE ID       CREATED        SIZE
# nginx         latest       a6bd71f48f68   2 weeks ago    188MB
# node          20-alpine    b8e6e2a3c1f1   3 days ago     177MB
# postgres      16           5d7c3e8e22f3   1 week ago     438MB

# Remove a single image
docker rmi nginx:latest

# Remove all unused images
docker image prune

# Remove ALL images (be careful!)
docker rmi $(docker images -q)
```

> **Warning:** `docker image prune` and `docker system prune` can free a lot of disk space, but they also delete images and containers you might still need. Read the prompt carefully before confirming.

Fixing the "Image Is Being Used" Error

A very common error when removing images looks like this. You try `docker rmi nginx:latest` and Docker refuses:

```
$ docker rmi nginx:latest
Error response from daemon: conflict: unable to remove
repository reference "nginx:latest" (must force) -
container eebd38aa4c91 is using its referenced image 6f8edba05e38
```

This is not a bug. Docker is protecting you: an image cannot be deleted while a container — even a **stopped** one — still depends on it. The fix is to deal with the container first. You have two options.

Option 1: Remove the container, then the image (recommended)

```
# 1. Find the container using the image
#    Use -a so STOPPED containers show up too
docker ps -a

# CONTAINER ID   IMAGE     STATUS                     NAMES
# eebd38aa4c91   nginx     Exited (0) 5 minutes ago   my-nginx

# 2. Remove that container (use -f if it is still running)
docker rm eebd38aa4c91
# or by name:
docker rm my-nginx

# 3. Now the image deletes cleanly
docker rmi nginx:latest
```

The container ID in your error message (here, `eebd38aa4c91`) is exactly the one you need to remove in step 2 — Docker tells you which container is the blocker.

Option 2: Force-remove the image

The `-f` (or `--force`) flag tells Docker to remove the image anyway. It does this by **untagging** the image — the underlying layers stay on disk as long as the container references them, and are cleaned up once that container is gone.

```
# Force-remove the image reference
docker rmi -f nginx:latest

# The container eebd38aa4c91 still exists and still runs,
# but the image is now "dangling" (untagged, shown as <none>)
docker images
# REPOSITORY   TAG       IMAGE ID       SIZE
# <none>       <none>    6f8edba05e38   161MB
```

Warning: Forcing removal does not actually free disk space while a container is still using the image — it only removes the name/tag. Prefer Option 1: remove the container first, then the image. That genuinely reclaims the space.

Tip: To clean up everything related to an image in one go: stop and remove all containers created from it, then remove the image. The one-liner `docker rm -f $(docker ps -aq --filter ancestor=nginx)` removes every container based on the `nginx` image, after which `docker rmi nginx` succeeds.

Image Tags and Versions

Tags are labels attached to an image that usually represent versions, variants, or build dates. The same image can have multiple tags pointing at it. Here are common patterns you'll see on Docker Hub:

- `latest` — the most recent stable build (avoid in production, see warning below).
- `20.10.0` — a specific semantic version, fully reproducible.
- `20-alpine` — major version on the Alpine Linux base.
- `20-slim` — a slimmer Debian variant with fewer pre-installed tools.
- `bullseye` or `bookworm` — built on a specific Debian release.

Warning: Avoid using `:latest` in production. It is a moving target — whoever pushes to the registry next gets to define what *latest* means. Pin to specific versions like `node:20.10.0-alpine` for reproducible builds.

Searching the Registry

You can search Docker Hub directly from the CLI, although the web interface at hub.docker.com usually gives a richer experience:

```
# Search for images named "redis"
docker search redis

# Limit results to top 5
docker search --limit 5 postgres

# Show only "official" images
docker search --filter is-official=true python
```

Tip: Look for images marked **Official Image** or **Verified Publisher** on Docker Hub. These are maintained by Docker or by the project itself and are far safer than random third-party uploads.

Image Layers Explained

Every image is built from a stack of **layers**. Each instruction in a Dockerfile produces a layer. Layers are cached and shared between images, which is what makes Docker so efficient. If two images both start with the same base layer (`FROM node:20-alpine`, for example), they share that layer on disk.

You can inspect the layers of any image:

```
# Show the history (layers) of an image
docker history nginx:latest

# Output (simplified):
# IMAGE          CREATED       CREATED BY                      SIZE
# a6bd71f48f68   2 weeks ago   CMD ["nginx" "-g" "daemon ...   0B
# <missing>      2 weeks ago   ENTRYPOINT ["/docker-entry...   0B
# <missing>      2 weeks ago   COPY ./docker-entrypoint.sh /   1.62kB
# <missing>      2 weeks ago   RUN /bin/sh -c set -x && add...  60.5MB
# <missing>      2 weeks ago   ENV NJS_VERSION=0.8.0           0B
```

Note: The order of instructions in a Dockerfile directly affects layer caching. Place rarely-changing instructions (like installing OS packages) before frequently-changing ones (like copying source code). We'll cover this in detail in Section 4.

3. Working with Containers

Starting Containers (docker run)

The `docker run` command is the workhorse of the Docker CLI. It creates a new container from an image and starts it. The general syntax is:

```
docker run [OPTIONS] IMAGE [COMMAND] [ARG...]
```

Here are the options you'll use over and over:

- `-d` or `--detach` — run in the background.
- `-it` — interactive + TTY, used for shells.
- `-p HOST:CONTAINER` — publish a port.
- `--name NAME` — assign a name.
- `-e KEY=VALUE` — set an environment variable.
- `-v HOST:CONTAINER` — mount a volume.
- `--rm` — remove the container automatically when it exits.
- `--network NAME` — attach to a specific network.

A few realistic examples:

```
# Run an interactive Ubuntu shell, removed on exit
docker run -it --rm ubuntu:22.04 bash

# Run Redis in the background, exposed on host port 6379
docker run -d -p 6379:6379 --name cache redis:7-alpine

# Run Postgres with a password and a named volume for data
docker run -d \
    --name db \
    -e POSTGRES_PASSWORD=secret \
    -v pgdata:/var/lib/postgresql/data \
    -p 5432:5432 \
    postgres:16
```

Detached Mode and Naming

By default `docker run` attaches your terminal to the container, which is useful for short-lived commands but not for long-running services. Adding `-d` runs the container in the background and prints its ID. Combine that with `--name` so you can refer to the container by a friendly handle.

```
# Bad: random name, you'll need the ID later
docker run -d nginx

# Good: predictable name, easy to script
docker run -d --name web nginx
```

Listing Containers (ps)

You can list running containers — or all containers — using:

```
# Show running containers
docker ps

# Show all containers, including stopped ones
docker ps -a

# Show only container IDs (useful in scripts)
docker ps -aq

# Filter by status
docker ps --filter "status=exited"

# Show container resource usage live
docker stats
```

A typical `docker ps` row contains the following fields. The real output is one wide line per container; here it is broken out field by field:

```
CONTAINER ID   4d2c8e5a1b03
IMAGE          nginx
COMMAND        "/docker-entrypoint..."
CREATED        10 minutes ago
STATUS         Up 10 minutes
PORTS          0.0.0.0:8080->80/tcp
NAMES          web
```

Stopping, Starting, Restarting

```
# Gracefully stop a container (sends SIGTERM, then SIGKILL after 10s)
docker stop web

# Start a stopped container
docker start web

# Restart (stop + start)
docker restart web

# Force-kill immediately (sends SIGKILL)
docker kill web

# Pause / unpause (freeze the process)
docker pause web
docker unpause web
```

> **Note:** `docker stop` is almost always what you want. `docker kill` is for unresponsive containers — it skips the graceful shutdown signal.

Removing Containers

```
# Remove a stopped container
docker rm web

# Force-remove a running container (stop + remove)
docker rm -f web

# Remove all stopped containers
docker container prune

# Remove every container on the system
docker rm -f $(docker ps -aq)
```

> **Tip:** If you don't need the container after it exits — for example, a one-off CLI tool — add `--rm` to `docker run` so cleanup happens automatically.

Executing Commands Inside a Container

The `docker exec` command runs an additional process inside an already-running container. The most common use is opening a shell to debug or inspect state:

```
# Open an interactive bash shell inside the "web" container
docker exec -it web bash

# Some minimal images don't have bash -- use sh instead
docker exec -it web sh

# Run a one-off command without a shell
docker exec web ls /etc/nginx

# Run a command as a different user
docker exec -u root -it web bash
```

Viewing Logs

Anything a container writes to **stdout** or **stderr** is captured by Docker and accessible via the logs command:

```
# Print all logs
docker logs web

# Follow logs in real time (like tail -f)
docker logs -f web

# Show only the last 100 lines
docker logs --tail 100 web

# Show logs with timestamps
docker logs -t web

# Show logs from the last 5 minutes
docker logs --since 5m web
```

Best Practice: Configure your application to log to stdout/stderr instead of files inside the container. This is the 12-factor approach and lets Docker (and orchestrators like Kubernetes) collect logs centrally.

Inspecting Containers

For deep introspection — IP address, mount points, environment, network settings, and more — use `docker inspect`:

```
# Print full JSON details of a container
docker inspect web

# Extract a specific field using a format template
docker inspect -f '{{ .NetworkSettings.IPAddress }}' web

# Get the exit code of a stopped container
docker inspect -f '{{ .State.ExitCode }}' web
```

4. Writing Dockerfiles

What Is a Dockerfile?

A Dockerfile is a plain text file containing the instructions Docker uses to build a custom image. Each instruction is on its own line and produces a new layer. Dockerfiles are the foundation of reproducible builds — anyone with the same file can produce the same image.

Anatomy of a Dockerfile

Here is a minimal example. We'll dissect every line below:

```
# Use an official Node.js LTS image as the base
FROM node:20-alpine

# Set the working directory inside the container
WORKDIR /app

# Copy dependency manifests first (better caching)
COPY package*.json ./

# Install production dependencies
RUN npm ci --omit=dev

# Copy the rest of the application source code
COPY . .

# Document the port the application listens on
EXPOSE 3000

# Define environment variables
ENV NODE_ENV=production

# Default command to run when the container starts
CMD ["node", "server.js"]
```

Common Instructions Explained

- `FROM` — every Dockerfile starts here. It sets the base image. Use a specific tag for reproducibility.
- `WORKDIR` — sets the working directory for following instructions and the container's default working directory.
- `COPY` — copies files from your build context into the image. Prefer COPY over the older ADD unless you need ADD's extras (URLs, auto-extract).
- `RUN` — executes a command during the build, such as installing packages. Each RUN creates a new layer.
- `ENV` — sets an environment variable in the image. Persists at runtime.
- `ARG` — defines a build-time variable. Disappears after the build.
- `EXPOSE` — documents which port(s) the container listens on. Doesn't actually publish anything.
- `USER` — sets the user the container runs as. Use a non-root user for security.

- `CMD` — the default command to run when a container starts. Can be overridden at runtime.
- `ENTRYPOINT` — like CMD, but harder to override. Typically used to define the executable.

> **Note: CMD vs ENTRYPOINT**: `CMD` is easy to override at `docker run` time — perfect for default arguments. `ENTRYPOINT` is fixed — use it when your image *is* a particular executable. They can be combined: ENTRYPOINT for the binary, CMD for default flags.

Your First Dockerfile (Node.js App)

Let's containerize a tiny Express app from scratch. By the end you'll have this directory layout:

```
my-app/
|-- package.json
|-- server.js
|-- .dockerignore
`-- Dockerfile
```

Step 1: Create the project folder

Open a terminal and create a new directory for the project:

```
# Create and move into the project folder
mkdir my-app
cd my-app
```

Step 2: Create the package.json file

The `package.json` file describes your Node.js project and its dependencies. Generate it with `npm init -y` -- the `-y` flag accepts all the defaults and creates the file instantly without asking any questions:

```
# Create package.json with default values, no prompts
npm init -y
```

Next, install Express. This downloads the package and automatically adds it to the `dependencies` section of `package.json`:

```
# Install Express and save it as a dependency
npm install express
```

After running those commands, open `package.json` in your editor and adjust it so it matches the following. The important parts are `main` (the entry file) and the `start` script that tells Node which file to run:

```
{
  "name": "my-app",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.19.0"
  }
}
```

Note: Running `npm install` also creates a `node_modules` folder and a `package-lock.json` file. You do **not** copy `node_modules` into the image -- the Dockerfile installs dependencies fresh inside the container. We exclude it with `.dockerignore` shortly.

Step 3: Create the server.js file

Create a file named `server.js` in the project root with a minimal Express server that responds on the root route:

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello from inside a Docker container!');
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Listening on port ${PORT}`);
});
```

Step 4: Create the Dockerfile

Now create a file named exactly `Dockerfile` (no extension) in the project root. This is the recipe Docker follows to build your image. Each line is explained in the comments:

```dockerfile
# Start from the official Node.js 20 image (Alpine = small)
FROM node:20-alpine

# Set the working directory inside the container
WORKDIR /app

# Copy ONLY the dependency manifests first (better caching)
COPY package*.json ./

# Install dependencies inside the container
RUN npm install --omit=dev

# Copy the rest of the application source code
COPY . .

# Document the port the app listens on
EXPOSE 3000

# The command that runs when the container starts
CMD ["node", "server.js"]
```

Step 5: Create the .dockerignore file

Finally, create a `.dockerignore` file so local files like `node_modules` are not copied into the image (this is covered in detail in the `.dockerignore` section below):

```
# .dockerignore
node_modules
npm-debug.log
.git
.env
```

> **Tip:** The file must be named `Dockerfile` with a capital D and **no file extension**. If your editor saves it as `Dockerfile.txt`, Docker won't find it automatically -- you'd have to point to it with `docker build -f`.

Building the Image

From the project root, run `docker build`: